Event Handling

```
@Override
public Event[] simulate() {
  boolean success = this.stop.addPassenger(this.passenger);
  if (!success) {
    return new Event[] {
      new DepartureEvent(this.getTime(), this.passenger)
    };
  }
  return new Event[] { };
}

@Override
public String toString() {
  return super.toString() + " " + this.passenger + " arrives at " + this.stop;
}
```

```
class Queue<T> implements Comparable<Queue<T>> {
    :

  @Override
  public int compareTo(Queue<T> q) {
    if (this.length() < q.length()) {
      return -1;
    }
    return 1;
  }
}
```

interface → only method is compareTo (remember to implement yourself)

```
public interface Comparable<T> {
    int compareTo(T other);
}
```

compareTo returns
< 0 -> this.object is smaller
0 -> objects are equal
> 0 -> this.object is larger

```
// QUEUE
enq(T e);
deq();
peek(); // does not remove it
isFull();
isEmpty();
length();
```

```
public class Seq<T extends Comparable<T>> {
  private T[] array;

  public Seq(int size) {
    @SuppressWarnings("unchecked")
    T[] temp = (T[]) new Comparable<?>[size];
    this.array = temp;
  }

  public void set(int index, T item) {
    if (index < this.array.length) {
      this.array[index] = item;
    }
  }

  public T get(int index) {
    return this.array[index];
  }

  public int size() {
    return this.array.length;
  }

  public T min() {
    T minimum = this.array[0];

    for (int i = 0; i < this.array.length; i++) {
      if (this.array[i].compareTo(minimum) < 0) {
        minimum = this.array[i];
      }
    }
    return minimum;
  }

  @Override
  public String toString() {
    StringBuilder s = new StringBuilder("[ ");
    for (int i = 0; i < this.array.length; i++) {
      s.append(i + ":" + this.array[i]);
      if (i != this.array.length - 1) {
        s.append(", ");
      }
    }
    return s.append(" ]").toString();
  }
}
```

Comparable<?> means "something that is comparable to something"

```
class Seq<T>{
 private T[] array;

 public Seq(int size){
  // The only way we can put an object into array is through
  // the method set() and we only put object of type T inside.
  // So it is safe to cast `Object[]` to `T[]`.
  @SuppressWarnings("unchecked")
  T[] a = (T[]) new Object[size];
  this.array = a;
 }
 public void set(int index, T item){
  this.array[index] = item;
 }
 public T get(int index){
  return this.array[index];
 }
}

class A{
 public static <S> boolean contains(Seq<? extends S> seq, S obj){
  for (int i = 0; i < seq.getLength(); i++){
   S curr = seq.get(i);
   if (curr.equals(obj)){
    return true;
   }
  }
  return false;
```

Wildcards

- **Upper bounded wildcards** are covariant, where S <: T `C<S>` <: `C<? extends S>` <: `C<? extends T>`
- **Lower bounded wildcards** are contravariant, where T >: S. `C<S>` <: `C<? super S>` >: `C<? super T>`
- **Unbounded wildcards** `<?>` are substituted with any type, where `C<S>`, `C<? extends S>`, and `C<? super S>` <: `C<?>`
- **PECS** (Producer Extends; Consumer Super): Producers (getters) return variables, consumers (setters) accept variables
- **Diamond operator** `new C<>()` is used for type inference when we instantiate a generic type

Wildcard	Meaning	Example Usage
?	**Unknown type**	List<?> (A list of some unknown type)
? extends T	**Some subclass of T** (read-only)	List<? extends Number> (Could be List<Integer>, List<Double>, etc.)
? super T	**Some superclass of T** (write-safe)	List<? super Integer> (Could be List<Integer>, List<Number>, or List<Object>)

```
public static <S> ArrayStack<S> of(S[] a, int maxStackSize) {
    ArrayStack<S> tempStack = new ArrayStack<>(maxStackSize);

    int copyLength = Math.min(a.length, maxStackSize);
    for (int i = 0; i < copyLength; i++) {
        tempStack.push(a[i]);
    }
    return tempStack;
}
```

The method `of` is called a *factory method*. A factory method is a method provided by a class for the creation of an instance of the class. Using a public constructor to create an instance necessitates calling `new` and allocating a new object on the heap every time. A factory method, on the other hand, allows the flexibility of reusing the same instance. The `of` method does not reuse instances. You will write another one that reuses available instances in the next section.

:[x] → jump to line x

gg → Go to the top of the file.
G → Go to the bottom of the file.
0 → Move to the beginning of the current line.
$ → Move to the end of the current line.

yy → Yank (copy) the current line.
y$ → Yank from cursor to end of line.
y0 → Yank from cursor to start of line.
p → Paste after the cursor.
P → Paste before the cursor.

dd → Delete (cut) the current line.
D → Delete from cursor to the end of the line.
d0 → Delete from cursor to the start of the line.
dG → Delete from the current line to the end of the file.
dgg → Delete from the current line to the top of the file.

u → Undo last change.

vsp[filename] - open the file in vertical split mode
cp filename filename2 (in terminal)

EXCEPTIONS
unchecked (programmer error) → RuntimeException
checked → Exception
creating → method sig. throws exception
throwing → throw new exception
try-catch-finally → try { normal } catch (Exception e) {...} finally { both cases execute this code }

```
public void popAll(ArrayStack<? super T> stack) {
    int copyLength = Math.min(
            stack.getStackSize(),
            maxStackSize - this.getStackSize() + 1);

    for (int i = 0; i < copyLength; i++) {
        stack.push(this.pop());
    }
}
```

<S super T> is not proper syntax as it would leave S too ambiguous.
Use wildcards instead!!

```
public void pushAll(ArrayStack<? extends T> stack) {
```

```
public <S extends T> void pushAll(ArrayStack<S> stack) { ... }
```

both of these options work, but ? is preferred for more flexibility

To compare outputs:

```
java PE1 < inputs/PE1.1.in > OUT
vim -d OUT outputs/PE1.1.out
```

```java
class Pair<S,T> {
  private S first;
  private T second;

  public Pair(S first, T second) {
    this.first = first;
    this.second = second;
  }

  public S getFirst() {
    return this.first;
  }

  public T getSecond() {
    return this.second;
  }
}
```

```java
class DictEntry<T> extends Pair<String,T> {
    :
}
```

```java
class A {
  public static <T> boolean contains(T[] array, T obj) {
    for (T curr : array) {
      if (curr.equals(obj)) {
        return true;
      }
    }
    return false;
  }
}
```

```java
String[] strArray = new String[] { "hello", "world" };
A.<String>contains(strArray, 123); // type mismatch error
```

```java
class A {
  public static <T extends GetAreable> T findLargest(T[] array) {
    double maxArea = 0;
    T maxObj = null;
    for (T curr : array) {
      double area = curr.getArea();
      if (area > maxArea) {
        maxArea = area;
        maxObj = curr;
      }
    }
    return maxObj;
  }
}
```

Java will strictly enforce whatever is in here

method declaration

Generics are invariant so if

```java
A.<Circle>contains(shapeSeq, circle); // compilation error
```

✔ Use wildcards (? extends T / ? super T) when we want to work with multiple types but can't list them all.
✔ Use generics (<T>) when we want strict type consistency throughout a method.
✔ Wildcards are more flexible, but generics are more type-safe.

Although you **can't use wildcards in class declarations,** you **can use them inside the class** in method parameters, return types, and fields.

Equipment.java

```java
abstract class Equipment {
  private boolean inUse = false;

  public boolean isInUse() {
    return this.inUse;
  }

  public void setInUse(boolean inUse) {
    this.inUse = inUse;
  }

  public abstract void repair();
}
```

Abstract classes do not need constructors if none of its fields have to be initialized (but you can have them regardless)

```java
class TimesOperation extends Operation {
    public TimesOperation(Expression x, Expression y) {
        super(x, y);
    }

    @Override
    public Object eval() {
        Object objX = this.getX().eval();
        Object objY = this.getY().eval();
        if (objX instanceof Integer && objY instanceof Integer) {
            int x = (Integer) objX;
            int y = (Integer) objY;
            return x * y;
        } else {
            throw new InvalidOperandException('*');
        }
    }
}
```

Creating generic arrays
↳ creating the array directly is not allowed, so you need to make a temporary array

```java
public class ArrayStack<T> implements Stack<T> {
    private int maxStackSize;
    private T[] stack;
    private int currentIndex = 0;

    public ArrayStack(int maxStackSize) {
        this.maxStackSize = maxStackSize;
        // I know what I am doing alright?
        @SuppressWarnings("unchecked")
        T[] temp = (T[]) new Object[maxStackSize];
        this.stack = temp;
    }
```

```java
public static <S> ArrayStack<S> of(S[] a, int maxStackSize) {
    ArrayStack<S> tempStack = new ArrayStack<>(maxStackSize);

    int copyLength = Math.min(a.length, maxStackSize);
    for (int i = 0; i < copyLength; i++) {
        tempStack.push(a[i]);
    }
    return tempStack;
}
```

```java
class JunctionSimulation extends Simulation {

  private Junction junction;

  public static final int CAR = 0;
  public static final int PEDESTRIAN = 1;

  public Event[] initEvents;

  public JunctionSimulation(Scanner sc) {
    initEvents = new Event[sc.nextInt() + 1];

    int numVehicleLanes = sc.nextInt();
    int vehicleLaneQueueSize = sc.nextInt();
    Road road = new Road(numVehicleLanes, vehicleLaneQueueSize);

    int pedestrianLaneQueueSize = sc.nextInt();
    Road crossing = new Road(1, pedestrianLaneQueueSize);

    this.junction = new Junction(road, crossing);

    int i = 0;
    while (sc.hasNextDouble()) {
      double arrivalTime = sc.nextDouble();
      int type = sc.nextInt();
      if (type == JunctionSimulation.CAR) {
        // TODO: create arrival event here
        initEvents[i] = new ArrivalEvent(new Car(), this.junction, arrivalTime);
      } else if (type == JunctionSimulation.PEDESTRIAN) {
        // TODO: create arrival event here
        initEvents[i] = new ArrivalEvent(new Pedestrian(), this.junction, arrivalTime);
      }
      i += 1;
    }
    // TODO: create signaling event here
    initEvents[i] = new CarSignalGreen(this.junction, 0);
  }

  @Override
  public Event[] getInitialEvents() {
    return initEvents;
  }
}
```